CLEAN CODE : Topics & Exercises

Clean Code means writing software that is easy to read, understand, and maintain. It focuses on clarity, simplicity, and good design principles. Clean code avoids duplication, follows consistent formatting, and expresses intent clearly. The main goal is to make code readable for other developers and future maintenance.

Example: Student Grade Calculator

```
import java.util.Scanner;
public class StudentGradeCalculator {
 public static char calculateGrade(int studentMarks) {
        if (studentMarks >= 90) return 'A';
        if (studentMarks >= 75) return 'B';
        if (studentMarks >= 60) return 'C';
        return 'D';
    }
public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter student marks (0-100): ");
        int studentMarks = scanner.nextInt();
        System.out.println(calculateGrade(studentMarks));
    }
}
```

Meaningful Names

Meaningful names improve readability and reduce confusion. Variables, classes, and methods should clearly describe their purpose. Good naming eliminates the need for unnecessary comments and helps developers quickly understand the logic of the program.

Example: Rectangle With Clean & meaningful Names

```
public class RectangleApp {
  public static void main(String[] args) {
      Rectangle rectangle = new Rectangle(10, 5);
      System.out.println("Area: " + rectangle.getArea());
   }
}
class Rectangle {
   private final double length;
   private final double width;
   public Rectangle(double length, double width) {
      if (length <= 0 || width <= 0) {
         throw new IllegalArgumentException("Length and width must be positive.");
      }
      this.length = length;
      this.width = width;
   }
   public double getArea() {
      return length * width;
   }
}
```

Functions

Functions should be small, focused, and perform only one task. A good function has a clear name, minimal parameters, and no hidden side effects. Small functions improve readability, testability, and maintainability of the system.

Example: Even or Odd Number Classification

```
import java.util.Scanner;
public class NumberClassification {


    public static boolean isEven(int number) {
        return number % 2 == 0;
    }
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int inputNumber = scanner.nextInt();
        if (isEven(inputNumber)) {
            System.out.println("The number is even.");
        } else {
            System.out.println("The number is odd.");
        }
        scanner.close();
    }
}
```

Comments

Comments should explain why something is done, not what is done. Good code is self-explanatory and does not need excessive comments. Outdated or unnecessary comments can create confusion, so they should be avoided.

Example: Simple Addition Calculator Using Clean Code Principles

```
import java.util.Scanner;
public class SimpleCalculator {
    public static int add(int firstNumber, int secondNumber) {
        return firstNumber + secondNumber;
    }
```

```
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter first number: ");
        int firstNumber = scanner.nextInt();
        System.out.print("Enter second number: ");
        int secondNumber = scanner.nextInt();
        int result = add(firstNumber, secondNumber);
        System.out.println("Sum: " + result);
        scanner.close();
    }
}
```

Formatting

Proper formatting improves code readability. Consistent indentation, spacing, and structure help developers understand logic quickly. Well-formatted code looks professional and organized, making collaboration easier.

Example : Employee Net Salary Calculator with Tax Deduction

```
import java.util.Scanner;
public class EmployeeSalaryCalculator {
    private static final double TAX_RATE = 0.10;
    public static double calculateNetSalary(double baseSalary) {
        if (baseSalary < 0) {
            throw new IllegalArgumentException("Base salary must be non-negative.");
        }
        double taxAmount = baseSalary * TAX_RATE;
        return baseSalary - taxAmount;
    }
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter base salary: ");
        double baseSalary = scanner.nextDouble();
```

```
        double netSalary = calculateNetSalary(baseSalary);
        System.out.println("Net Salary: " + netSalary);
        scanner.close();
    }
}
```

Objects and Data Structures

Objects should hide internal data and expose behavior through methods. Data structures should clearly represent data without unnecessary complexity. Encapsulation ensures that internal details are protected and modified only through controlled methods.

Example : Student Object with Proper Encapsulation

```
public class Student {
    private final String name;
    private final int age;
    public Student(String name, int age) {
        validateAge(age);
        this.name = name;
        this.age = age;
    }
    private void validateAge(int age) {
        if (age <= 0) {
            throw new IllegalArgumentException("Age must be greater than zero.");
        }
  }
    public String getName() {
        return name;
    }
    public int getAge() {
        return age;
    }  }
```

Student Application (Using the Student Object):

```
import java.util.Scanner;
public class StudentApplication {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter student name: ");
        String name = scanner.nextLine();
        System.out.print("Enter student age: ");
        int age = scanner.nextInt();
        Student student = new Student(name, age);
        System.out.println("Student Name: " + student.getName());
        System.out.println("Student Age: " + student.getAge());
        scanner.close();
    }
}
```

Error Handling

Good error handling separates normal logic from error logic. Exceptions should be used instead of error codes to improve clarity. Error messages should be meaningful and help in debugging without exposing unnecessary internal details.

Example : Safe Division Handler with Proper Error Handling

```
import java.util.Scanner;
public class DivisionHandler {
    public static int divide(int dividend, int divisor) {
        validateDivisor(divisor);
        return dividend / divisor;
    }
    private static void validateDivisor(int divisor) {
        if (divisor == 0) {
            throw new IllegalArgumentException("Divisor must not be zero.");
```

```
        }
    }
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter dividend: ");
        int dividend = scanner.nextInt();
        System.out.print("Enter divisor: ");
        int divisor = scanner.nextInt();
        int result = divide(dividend, divisor);
        System.out.println("Result: " + result);
        scanner.close();
    }
}
```

Boundaries

Boundaries define how our code interacts with external systems or libraries. Using interfaces and abstraction helps isolate third-party code and makes the system flexible and easy to test. Proper boundary design reduces dependency issues.

Example : Create a payment system that depends on an external payment gateway. Isolate it using an interface.

```
interface PaymentGateway {
    void processPayment(double amount);
}
class StripeGateway implements PaymentGateway {
    public void processPayment(double amount) {
        System.out.println("Processing payment of " + amount);
    }
}
```

```
class PaymentService {
    private final PaymentGateway gateway;
    public PaymentService(PaymentGateway gateway) {
        this.gateway = gateway;
    }
    public void makePayment(double amount) {
        gateway.processPayment(amount);
    }
}
```

Unit Tests

Unit tests verify that small parts of the system work correctly. Good tests are clear, independent, and fast. They help prevent bugs, support refactoring, and improve overall software quality.

Example : Write a test for a discount calculator

```
class DiscountCalculator {
    public double applyDiscount(double price) {
        return price * 0.9;
    }
}


import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;
class DiscountCalculatorTest {
    @Test
    void shouldApplyTenPercentDiscount() {
        DiscountCalculator calculator = new DiscountCalculator();
        double finalPrice = calculator.applyDiscount(100);
        assertEquals(90, finalPrice);
    }
}
```

Classes

Classes should follow the Single Responsibility Principle, meaning they should have only one reason to change. Small and focused classes improve system design and reduce complexity.

Example :

1.Employee Class (Holds Data) :

Responsibility: Store employee information only.

```
public class Employee {
    private final String name;
    private final double monthlySalary;
    public Employee(String name, double monthlySalary) {
        this.name = name;
        this.monthlySalary = monthlySalary;
    }
    public String getName() {
        return name;
    }
    public double getMonthlySalary() {
        return monthlySalary;
    }
}
```

2.SalaryCalculator Class (Single Responsibility):

Responsibility: Only calculate annual salary.

```
public class SalaryCalculator {
    public double calculateAnnualSalary(Employee employee) {
        return employee.getMonthlySalary() * 12;
    }
}
```

3. Main Class (Usage Example):

```
public class Main {
    public static void main(String[] args) {
        Employee employee = new Employee("Nikitha", 50000);
        SalaryCalculator salaryCalculator = new SalaryCalculator();
        double annualSalary = salaryCalculator.calculateAnnualSalary(employee);
        System.out.println("Employee: " + employee.getName());
        System.out.println("Annual Salary: " + annualSalary);
    }
}
```

Employee stores data. SalaryCalculator only calculates salary. If salary rules change, only SalaryCalculator changes. If employee data changes, only Employee changes. That means each class has one reason to change.

Emergence :

Good design emerges through continuous refactoring. Removing duplication, improving clarity, and simplifying code gradually leads to a clean and maintainable system.

Example: ✕ Step 1: Initial (Bad Design – Duplicate Code)

```
public class OrderService {
    public double calculateRegularCustomerPrice(double price) {
        double discount = price * 0.10;
        double finalPrice = price - discount;
        return finalPrice;
    }
    public double calculatePremiumCustomerPrice(double price) {
        double discount = price * 0.20;
        double finalPrice = price - discount;
        return finalPrice;
    }  }
```

Problems:

- Duplicate logic
- Repeated calculation steps

Step 2: First Refactoring (Remove Duplication): We extract common logic.

```
public class OrderService {
    public double calculateFinalPrice(double price, double discountRate) {
        double discount = price * discountRate;
        return price - discount;
    }
    public double calculateRegularCustomerPrice(double price) {
        return calculateFinalPrice(price, 0.10);
    }
    public double calculatePremiumCustomerPrice(double price) {
        return calculateFinalPrice(price, 0.20);
    }
}
```

Step 3: Further Simplification (Cleaner Design Emerges): We simplify even more.

```
public class OrderService {
    public double calculateFinalPrice(double price, boolean isPremiumCustomer) {
        double discountRate = isPremiumCustomer ? 0.20 : 0.10;
        return price - (price * discountRate);
    }
}
```

Now: Small, Clear, No duplication, Easy to maintain

Emergence means we don’t try to design everything perfectly at the start. We improve the design continuously by refactoring until a clean structure naturally appears.

Concurrency

Concurrency deals with executing multiple tasks simultaneously. Proper synchronization and thread safety are important to avoid data inconsistency and race conditions. Clean concurrent code avoids shared mutable state when possible.

Example : Thread-Safe Counter Using AtomicInteger

```
import java.util.concurrent.atomic.AtomicInteger;
public class Main {
    public static void main(String[] args) throws InterruptedException {
        Counter counter = new Counter();
        Thread thread1 = new Thread(() -> {
            for (int i = 0; i < 1000; i++) {
                counter.increment();
            }
        });
        Thread thread2 = new Thread(() -> {
            for (int i = 0; i < 1000; i++) {
                counter.increment();
            }
        });
        thread1.start();
        thread2.start();

        thread1.join();
        thread2.join();
        System.out.println("Final Count: " + counter.getCount());
    }
}
```

```
class Counter {
    private final AtomicInteger count = new AtomicInteger(0);
    public void increment() {
        count.incrementAndGet();   // Thread-safe operation
    }
    public int getCount() {
        return count.get();
    }
}
```

This program demonstrates safe concurrency using AtomicInteger. Two threads increment a shared counter 1000 times each. Since the counter is shared mutable state, we use AtomicInteger to ensure atomic operations and prevent race conditions. We also use join() to ensure the main thread waits until both threads complete before printing the final result.

Successive Refinement

Successive refinement means improving code step by step. Instead of writing perfect code at once, developers continuously refactor and enhance clarity and structure.

✕ Step 1 – Initial Working Code (Not Clean)

```
public class PriceCalculator {
    public double calculate(double price, boolean isPremium) {
        if (isPremium == true) {
            return price - (price * 0.20);
        } else {
            return price - (price * 0.10);
        }
    }
}
```

Problems:

- Method name not clear
- isPremium == true unnecessary
- Magic numbers (0.20, 0.10)
- Logic not very readable

Step 2 – Improve Naming & Remove Unnecessary Code

```
public class PriceCalculator {
    public double calculateFinalPrice(double price, boolean isPremiumCustomer) {
        if (isPremiumCustomer) {
            return price - (price * 0.20);
        }
        return price - (price * 0.10);
    }
}
```

Improvements:

- Better method name
- Better variable name
- Removed == true
- Cleaner structure

Step 3 – Remove Duplication & Magic Numbers

```
public class PriceCalculator {
    private static final double PREMIUM_DISCOUNT = 0.20;
    private static final double REGULAR_DISCOUNT = 0.10;
    public double calculateFinalPrice(double price, boolean isPremiumCustomer) {
   double discountRate = isPremiumCustomer ? PREMIUM_DISCOUNT : REGULAR_DISCOUNT;


        return price - (price * discountRate);
    }
}
```

Improvements:

- Removed magic numbers
- Single calculation logic
- Cleaner structure
- Easy to modify later

Smells and Heuristics

Code smells are warning signs of poor design, such as long methods, large classes, and duplicate code. Heuristics are guidelines to identify and fix these issues to maintain clean and high-quality software.

Example : Code Smell Version (Long Method)

```
import java.util.Scanner;

public class Report {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter marks out of 500: ");

        int marks = scanner.nextInt();



        int percentage = (marks * 100) / 500;

        System.out.println("Percentage: " + percentage);

        if (percentage >= 50) {

            System.out.println("Result: Pass");

        } else {

            System.out.println("Result: Fail");

        }

        scanner.close();
```

```
    }
}
```

Why This Is a Code Smell?

- main() does too many things
  - Takes input
  - Calculates percentage
  - Decides pass/fail
  - Prints output
- Hard to test logic separately
- Not clean design

Refactored Version (Clean Code)

```
import java.util.Scanner;
public class Report {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter marks out of 500: ");
        int marks = scanner.nextInt();
        int percentage = calculatePercentage(marks);
        String result = getResult(percentage);
        System.out.println("Percentage: " + percentage);
        System.out.println("Result: " + result);

        scanner.close();
    }
    public static int calculatePercentage(int marks) {
        return (marks * 100) / 500;
    }
    public static String getResult(int percentage) {
        return percentage >= 50 ? "Pass" : "Fail";
    }   }
```

Design Patterns

Design Patterns are common solutions to common software design problems.

When developers build software, they often face similar types of problems again and again — like:

- How to create objects properly?
- How to organize classes?
- How to manage communication between objects?

Instead of solving these problems from scratch every time, experienced developers created standard reusable solutions. These standard solutions are called Design Patterns.

- A design pattern is **not ready-made code**.
- It is a **concept or template** that tells you how to structure your code to solve a problem in a better way.

Types of Design Patterns

Design patterns are divided into **three main types**:

1.Creational Patterns : These deal with object creation.

Examples: Singleton, Factory Method, Prototype

These patterns control how objects are created to make the system flexible.

2.Structural Patterns : These deal with class and object structure.

Examples: Adapter, Composite

These patterns help build large structures using small objects.

3.Behavioral Patterns : These deal with communication between objects.

These patterns define how responsibilities are shared between objects.

Model-View-Controller

What Is MVC (Model–View–Controller)?

MVC is a **software architectural pattern** used to structure applications in a way that separates concerns and improves maintainability, scalability, and testability. In MVC, the application is logically split into three interconnected components:

1. **Model** – Manages the application's data, rules, and logic.
2. **View** – Handles the user interface (UI) and presentation of data.
3. **Controller** – Interprets user input and coordinates between the Model and View.

The key idea behind MVC is **separation of concerns** — each component has a specific role, making the system easier to modify without breaking other parts.

Model

- Represents the **core data and business logic** of an application.
- Handles operations like data retrieval, storage, updates, and rules validation.
- It *never* displays anything — it simply *knows* the data and rules.

Example: In an e-commerce site, the Product model contains product name, price, stock, and methods like updateStock().

2. View

- Responsible for **presentation** — what the user sees on the screen.
- Uses data from the Model and displays it in forms, tables, dashboards, etc.
- Doesn't perform logic or modify data itself.

Example: A product listing page that shows name, price, and image for each product.

3. Controller

- Acts as a **bridge** between user interaction and model data.
- Receives input (e.g., clicks, form submissions), validates it, and tells the Model what to do.
- Based on Model output, it selects the appropriate view to display.

Example: A ProductController that handles requests like "show all products" or "add new product".

How MVC Works — Sequence of Interaction

A typical MVC request flow:

1. **User interacts with the View** (e.g., clicks a button).
2. The **Controller receives the input**, processes it, and requests changes in the Model.
3. **Model updates** its data state and notifies any observers.
4. The **View pulls updated data** and re-renders the UI.

Domain- Driven – Design (DDD)

Domain-Driven Design (DDD) is a **software design approach** that helps developers build complex software by closely aligning the *software's structure and language* with the *business domain* it is meant to serve.

Core Ideas

Focus on the Domain The domain means the specific field or business problem the software aims to address (e.g., banking, insurance, e-commerce). DDD puts this at the center of the design.

Collaborate with Experts Developers work closely with domain experts (people who understand the business) to build a conceptual model that reflects real business needs.

Use a Ubiquitous Language Everyone (developers, business teams, stakeholders) uses the same language when talking about concepts. This shared vocabulary is called the ubiquitous language and is used in code, documentation, and discussions.

Split Big Systems into Bounded Contexts Instead of one giant model, DDD divides the software into bounded contexts — smaller parts that each have their own model and language.

How It Affects Code

In DDD, code elements like classes, methods, and variables are named based on the *domain language*. For example, in banking software you'd see things like "Account," "Customer," or "ProcessLoan," rather than generic technical names.

DDD is especially helpful when:

- The **business domain is complex**
- Requirements change often
- Good communication between business and tech teams is key
- You want software design that truly reflects business rules and processes

It's generally **not necessary** for small or simple applications.